

École nationale supérieure des Mines de Saint-Etienne

LaTeXProject
Rapport

Projet ASCON - Conception d'un Système Numérique

Élèves :
Samy TALEB

Encadrants :
RIGAUD Jean-Baptiste

18 décembre 2024

Table des matières

1 Introduction du projet **2**

2 Description générale de l'algorithme de chiffrement ASCON-128 **3**
2.1 Point historique 3
2.2 Principe de fonctionnement 3
2.3 Intérêt du langage SystemVerilog 4

3 Conception de l'algorithme de chiffrement ASCON-128 **4**
3.1 Environnement virtuel de travail 4
3.2 Structure de l'algorithme 6
3.2.1 L'addition de constante 6
3.2.2 La substitution 7
3.2.3 La diffusion linéaire 8
3.2.4 Permutation complète 8
3.2.5 Machine d'états 13
3.2.6 Implémentation de l'architecture complète 15

4 Difficultés liées au projet **18**

5 Conclusion **18**

1 Introduction du projet

Ce projet de conception de l'algorithme de chiffrement ASCON-128 s'inscrit dans mon parcours d'étudiant en ingénierie informatique et micro-électronique aux Mines de Saint-Etienne cycle ISMIN. Il permet de souligner l'importance de la sécurité dans ces domaines, notamment pour la protection des systèmes embarqués en micro-électronique. En effet, tous ces systèmes peuvent parfois contenir des informations confidentielles (informations bancaires, militaires, etc) et il est donc primordial de garantir leur sécurité contre les menaces potentielles. Nous utiliserons ainsi le langage de codage SystemVerilog pour développer une solution pour contrer ce problème : il s'agit de l'algorithme de chiffrement ASCON-128.
Le but principal de ce projet est de comprendre le fonctionnement de l'algorithme et d'apprendre à utiliser le langage de description SystemVerilog, très utile dans ce type de projets. Pour se faire, nous allons concevoir une machine d'états pour l'algorithme ASCON-128, puis simuler chaque composant de cette machine d'états et enfin analyser les résultats afin de garantir le bon fonctionnement du système. Cela nous offrira une vision du fonctionnement de l'algorithme ASCON-128 et de son efficacité en terme de chiffrement d'informations.

2 Description générale de l'algorithme de chiffrement ASCON-128

2.1 Point historique

L'algorithme de chiffrement ASCON-128 a été développé par Christoph Dobraunig, Maria Eichlseder, Florian Mendel et Martin Schläffer en 2014. Il a été conçu dans le cadre du concours CAESAR (Competition for Authenticated Encryption : Security, Applicability, and Robustness) qui visait à promouvoir la création d'algorithmes de chiffrement. ASCON-128 a été sélectionné comme finaliste du concours CAESAR, ce qui a fait de lui un algorithme très reconnu dans le domaine du chiffrement de données et d'informations.

2.2 Principe de fonctionnement

Le principe de fonctionnement de l'algorithme de chiffrement ASCON-128 est assez complexe. Afin de mieux comprendre ses spécificités, il pourrait être utile de mettre en contexte le problème. Prenons le cas de Bob et Alice qui vivent éloignés l'un de l'autre et communiquent régulièrement par Internet. Ils souhaitent garder leur discussion secrète vis-à-vis d'Ève, susceptible d'espionner leurs conversations. Ils décident donc de s'envoyer des messages qu'ils seront les seuls à comprendre, même s'ils sont interceptés par Ève. Pour se faire, ils utiliseront l'algorithme ASCON-128 pour le chiffrement de leurs messages. Cet algorithme se divise en 4 temps : une phase d'initialisation de l'état, une phase de traitement des données associées (A), une phase de traitement du texte en clair (P) et enfin une phase de finalisation. Intéressons-nous à chacune d'entre-elles une à une :

— Initialisation de l'état S :
Avant de commencer à échanger des messages, Bob et Alice partagent une clé secrète (K) entre eux, codée sur 128 bits. Ils choisissent également un nombre arbitraire appelé "nonce" (N) pour chaque message qu'ils envoient, également codé sur 128 bits, pour garantir l'unicité et la sécurité de chacune des communications. Ensuite, l'état initial de l'algorithme (S) est initialisé en fonction de la clé secrète, du nonce et d'autres paramètres.

— Traitement des données associées (A) :
Bob et Alice peuvent inclure des données associées (A) dans leurs messages, comme des informations sur l'expéditeur ou la date de l'envoi. Ces données sont transformées en un format spécifique et sont intégrées au processus de chiffrement pour renforcer la sécurité de leur communication.

— Traitement du texte en clair (P) :
Lorsque Bob souhaite envoyer un message à Alice, il divise le message en blocs de 64 bits et les passe à travers l'algorithme ASCON-128. Chaque bloc de texte est mélangé avec l'état actuel de l'algorithme en utilisant une permutation spécifique appelée "p^6", qui effectue 6 rondes de l'unité de permutation p (qui sera détaillée plus tard). Cela produit un texte chiffré qui ne peut être compris que par Bob et Alice grâce à leur clé secrète.

— Finalisation :
Une fois que tous les blocs de texte ont été chiffrés, Bob envoie le message chiffré à Alice. Alice utilise la même clé secrète et le même processus de chiffrement pour déchiffrer le message. Elle applique également une permutation spécifique appelée "p^{12}", qui effectue 12 rondes de l'unité de permutation p. Elle peut également vérifier l'authenticité du message en calculant le tag T correspondant à partir de A et du texte chiffré (C).

On peut regrouper ces étapes sur le schéma suivant :

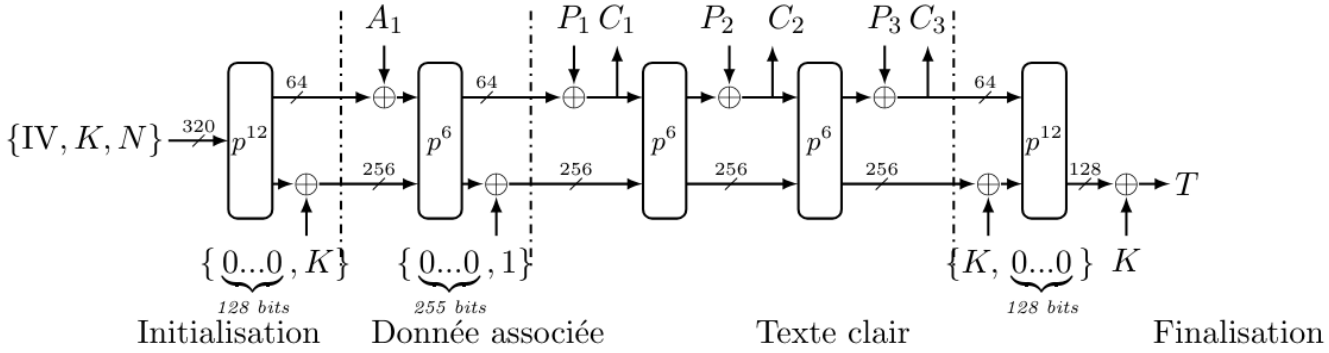


FIGURE 1 – Schéma du chiffrement ASCON-128

2.3 Intérêt du langage SystemVerilog

Le langage SystemVerilog est un langage de description matérielle fortement utilisé dans l'industrie électronique pour la conception de circuits numériques. Tout bon microéléctronicien se doit de maîtriser ce langage. Néanmoins, au delà de la nécessité d'avoir cette maîtrise, réaliser l'algorithme ASCON-128 en langage SystemVerilog permet de valider les circuits numériques créés à travers des simulations sur des logiciels compatibles avec ce langage. On pourra notamment simuler le fonctionnement de notre algorithme via le logiciel ModelSim, qui nous permettra d'obtenir le résultat de notre chiffrement et la valeur des sorties numériques associées à l'algorithme.

3 Conception de l'algorithme de chiffrement ASCON-128

3.1 Environnement virtuel de travail

La conception de l'algorithme de chiffrement ASCON-128 constitue un processus complexe qui fait appel à de nombreux fichiers de code. Etant donné leur grand nombre, il faut veiller à bien les organiser en définissant un emplacement pour chaque type de fichier. Ainsi, on distinguera les fichiers sources (contenant les instructions et les codes nécessaires pour que l'algorithme fonctionne correctement) et les fichiers de librairie. Par la suite, on distinguera dans un deuxième temps les fichiers à compiler des fichiers de type testbench (fichiers servant à tester et vérifier les circuits numériques qui ont été conçus).

On peut observer ci-dessous l'organigramme représentatif de l'arborescence de l'algorithme ASCON-128 :

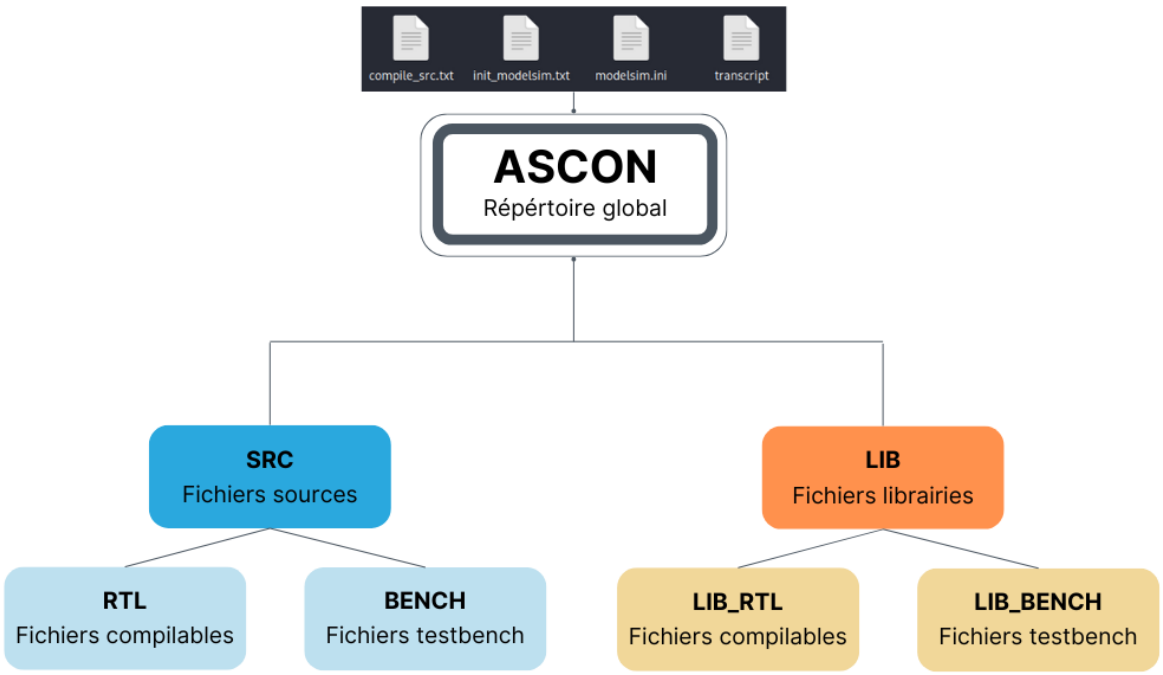


FIGURE 2 – Répertoire de travail du projet ASCON-128

On utilisera également la syntaxe suivante à travers nos codes :

1. Les entrées d'un module sont complétées par le suffixe "_i".
2. Les sorties d'un module sont complétées par le suffixe "_o".
3. Les signaux internes sont complétés par le suffixe "_s".

Pour compiler l'algorithme ou les fonctions intermédiaires, on se contentera d'exécuter le fichier de compilation compile_src.txt, en mettant en commentaire les fichiers que l'on ne souhaite pas compiler ou simuler. On utilisera donc les commandes suivantes dans le répertoire ASCON :

```
bash
source init_modelsim.txt
source compil_src.txt
```

3.2 Structure de l'algorithme

Comme nous l'avons dit précédemment, l'algorithme de chiffrement ASCON-128 constitue un processus complexe, il faut donc décomposer ce processus afin de mieux le réaliser. Reprenons le schéma de chiffrement de la partie 2.2 avec les 4 différentes étapes du processus de chiffrement. On peut dans un premier temps distinguer les XOR en début de permutation (xor_begin) des XOR en fin de permutation (xor_end). Cela nous aidera dans la suite de la réalisation de l'algorithme.

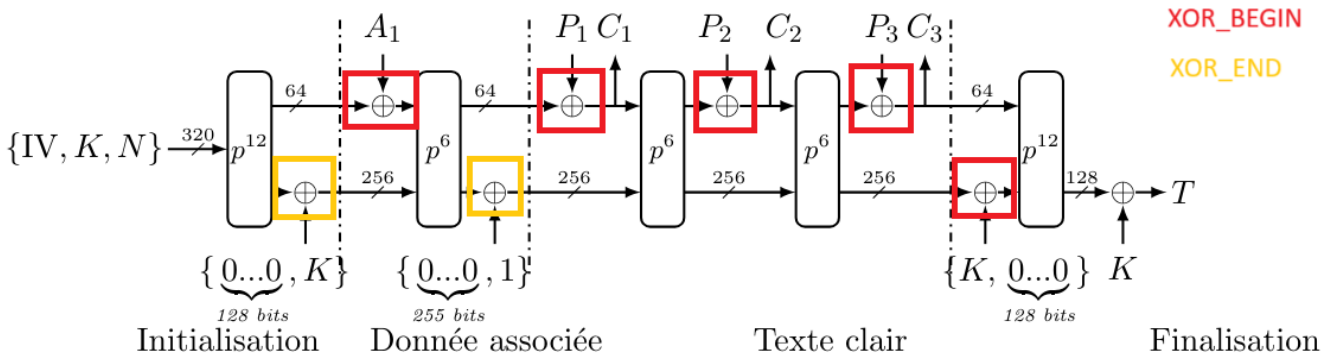


FIGURE 3 – Algorithme complet avec les XOR_begin en rouge et les XOR_end en jaune

Ensuite, il faut dans un second temps définir la ronde p qui définit les permutations p^6 et p^{12}. Chaque ronde se décompose en 3 opérations élémentaires : l'ajout de constante, la substitution et la diffusion linéaire. Ce sont les opérations clées à la base du chiffrement et auxquelles nous allons nous intéresser à présent.

3.2.1 L'addition de constante

L'addition de constante est la première transformation élémentaire qui compose une ronde de permutation p. Cette opération consiste à appliquer une modification sur le registre x_2 de l'état S en y ajoutant la constante de ronde associée à la ronde i. La constante se lit sur le tableau fourni dans le sujet (on notera que la présence de différentes constantes dans le tableau renforce le chiffrement de l'algorithme) :

Ronde r de p^{12}	Ronde r de p^6	Constante c_r
0		0000000000000000f0
1		0000000000000000e1
2		0000000000000000d2
3		0000000000000000c3
4		0000000000000000b4
5		0000000000000000a5
6	0	0000000000000000096
7	1	0000000000000000087
8	2	0000000000000000078
9	3	0000000000000000069
10	4	000000000000000005a
11	5	000000000000000004b

FIGURE 4 – Tableau de constante pour l'addition de constante

Après avoir implémenter cette transformation en langage SystemVerilog, le fichier testbench associé nous permet de réaliser la simulation. On obtient les résultats suivants pour les valeurs initiales du sujet :

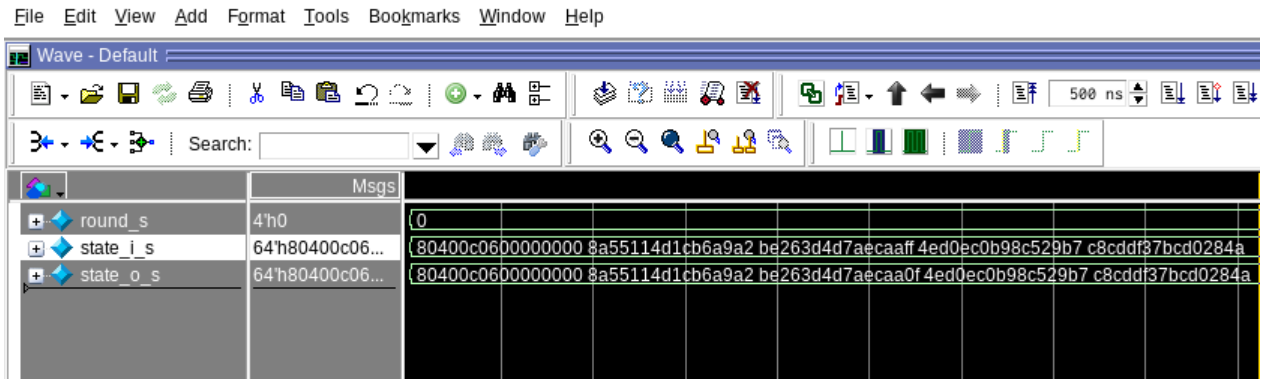


FIGURE 5 – Résultat de simulation de l'addition de constante

On obtient bien les résultats souhaités avec les mêmes sorties que celles attendues dans le sujet. On peut également bien remarquer que le XOR a été effectué sur x_2, ce qui est cohérent avec les instructions du sujet.

3.2.2 La substitution

La transformation élémentaire qui suit l'addition de constante est la substitution. Cette transformation se base des données d'un fichier appelé la *sbox*. Cette *sbox* permet de réaliser une substitution, donnée par le tableau suivant :

x	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
$S(x)$	04	0B	1F	14	1A	15	09	02	1B	05	08	12	1D	03	06	1C	1E	13	07	0E	00	0D	11	18	10	0C	01	19	16	0A	0F	17

FIGURE 6 – Table de substitution de la sbox

En implémentant la *sbox* en SystemVerilog et en lui associant un testbench avec les bonnes valeurs associées, on retrouve bien la table de substitution de la *sbox*. On peut l'observer sur la simulation ci-dessous :

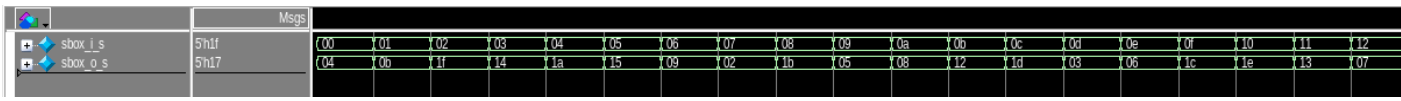


FIGURE 7 – Résultat de simulation de la sbox

La fonction de substitution consiste à concaténer les différentes valeurs obtenues par la table précédente pour chacun des bits de l'état, allant de 0 à 63 (soit 64 bits). En effet, chaque état de 320bits peut se décomposer en 5 blocs de 64bits et la fonction de substitution va ainsi appliquer la substitution de la *sbox* à chacun de ses blocs de 64bits.

En implémentant la fonction et en réalisant le testbench, on obtient les résultats suivants pour les valeurs en sortie de l'addition de constante :

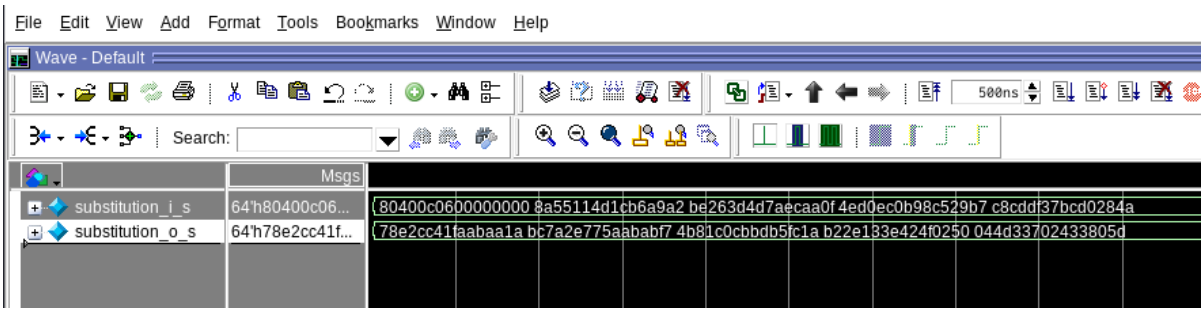


FIGURE 8 – Résultat de simulation de la substitution

On obtient bien les mêmes résultats que ceux attendus dans le sujet ce qui confirme que la transformation est bien implémentée.

3.2.3 La diffusion linéaire

La dernière transformation élémentaire est la diffusion linéaire. Cette transformation effectue une rotation cyclique qui correspond à un décalage de bits vers la droite. Voici les résultats de la simulation de cette transformation pour les valeurs en sortie de la substitution :

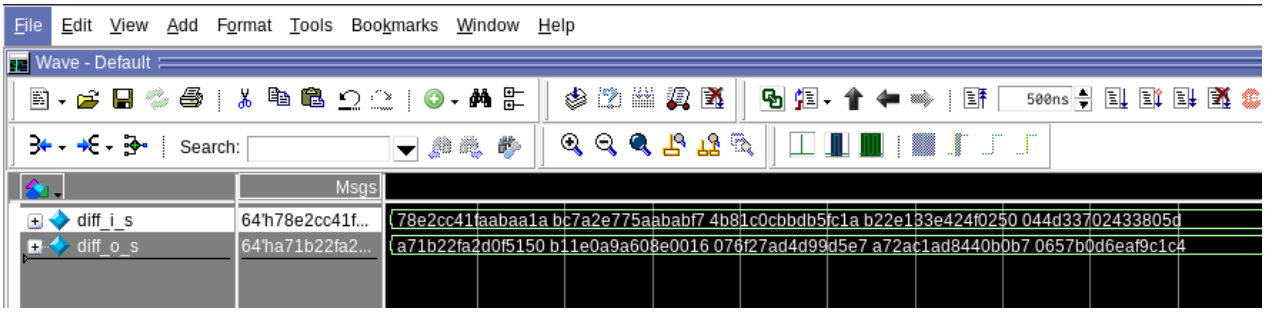


FIGURE 9 – Résultat de simulation de la diffusion linéaire

Une fois de plus, on obtient la bonne sortie qui correspond au résultat des 3 opérations élémentaires que l'on vient de détailler.

3.2.4 Permutation complète

Les 3 transformations élémentaires que l'on a détaillées précédemment sont essentielles pour l'implémentation d'une ronde p qui constitue les permutations p^6 et p^{12}. Cependant, une fois que ces 3 transformations qui composent une ronde ont bien été définies, il est nécessaire d'enregistrer l'état à la sortie de chaque ronde pour itérer les rondes 6 fois pour p^6 et 12 fois pour p^{12}. Il faut donc définir une mémoire d'état, reg_state, qui enregistrera l'état en fin de ronde. Le bloc de mémoire d'état se présentera donc de la sorte :

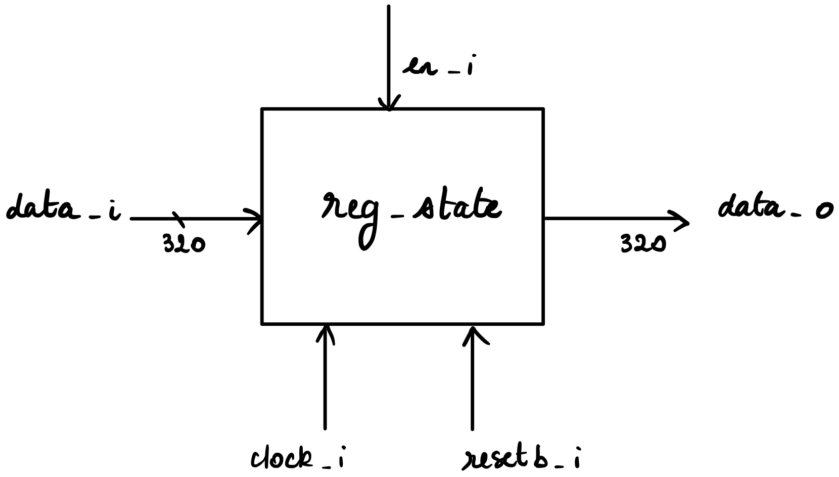


FIGURE 10 – Bloc *reg_state* de mémoire d'état

On va également ajouter un multiplexeur en début de permutation, permettant de choisir l'entrée de la permutation pour l'état qui sort de la ronde précédente. On obtient grâce au bloc *reg_state* et au multiplexeur une première version de la permutation qui est la suivante :

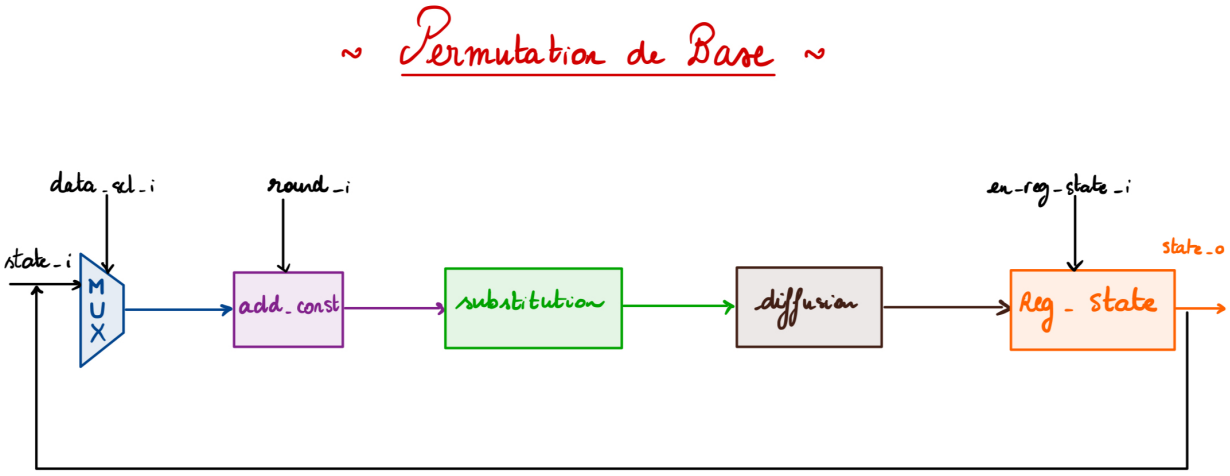


FIGURE 11 – Schéma de la permutation simple

Néanmoins, notre permutation n'est pas encore complète, mais presque ! Il reste à considérer les différents XOR, présents en début et de fin de permutation. En effet, comme nous l'avons évoqué lorsque l'on discutait de la structure et des étapes de l'algorithme de chiffrement, il existe deux type de XOR :

— Les XOR_begin sont présents avant les permutations (p^6 ou p^{12}). Ils peuvent être représentés par le bloc suivant, où l'on peut observer les entrées et sorties que ces XOR_begin comprennent :

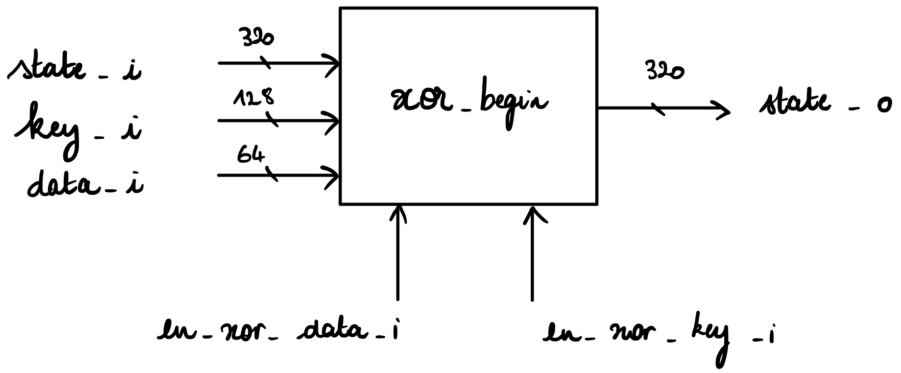


FIGURE 12 – Bloc représentatif d'un XOR_begin

On peut les voir en rouge sur le schéma suivant :

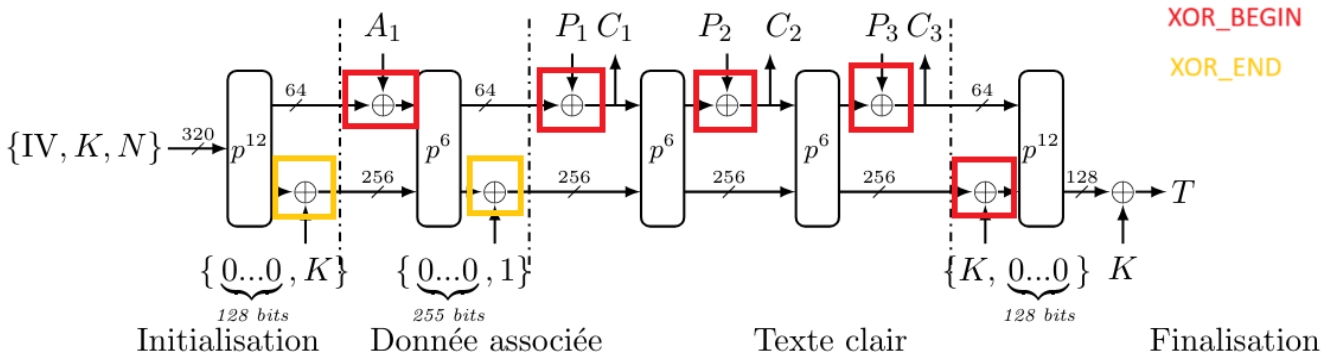


FIGURE 13 – Algorithme complet avec les XOR_begin en rouge et les XOR_end en jaune

— Les XOR_end sont présents après les permutations (p^6 ou p^{12}). Ils peuvent être représentés par le bloc suivant, où l'on peut observer les entrées et sorties que ces XOR_end comprennent :

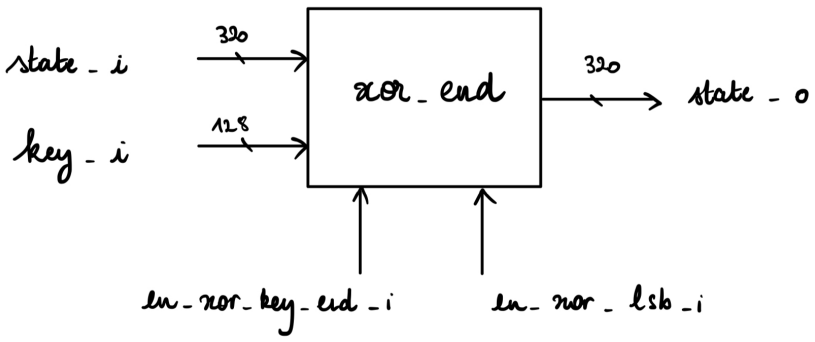


FIGURE 14 – Bloc représentatif d'un XOR_end

Ainsi, en tenant compte de ces XOR et en repartant de la permutation simple que l'on a définie précédemment, on obtient donc la permutation finale complète comme ci-dessous :

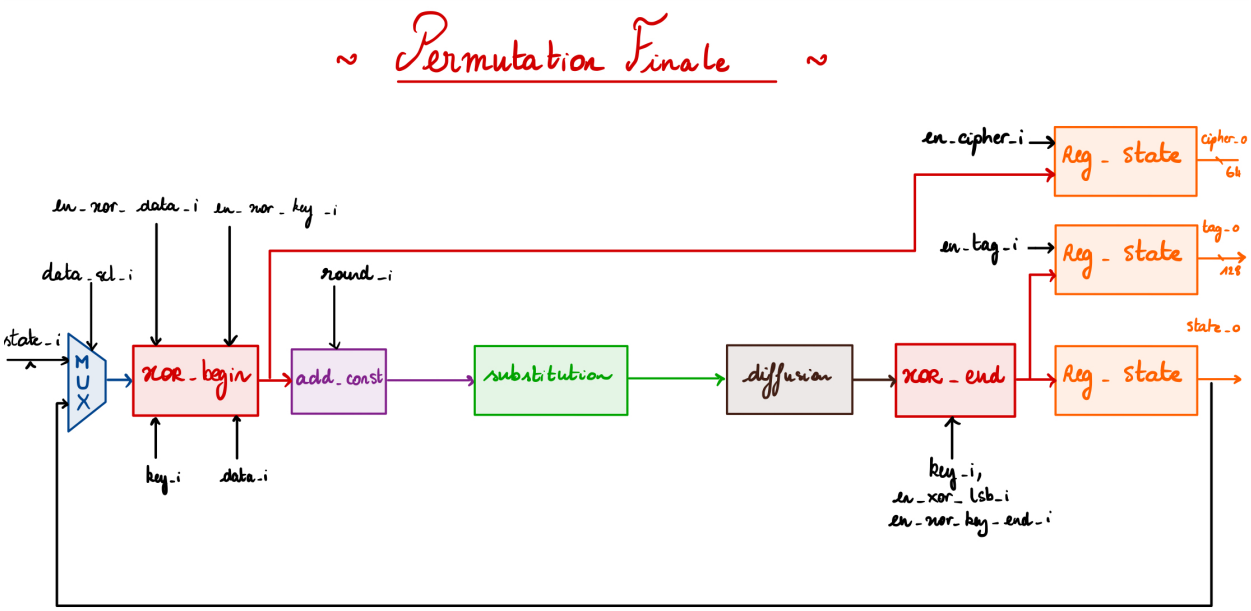


FIGURE 15 – Schéma de la permutation finale complète

Maintenant que la permutation finale complète a bien été définie, on peut l'implémenter dans un fichier SystemVerilog qui instancie chacun de ses blocs et établie des liaisons entre eux. On peut également réaliser un testbench associé à cette permutation finale complète pour la simuler et observer ses résultats pour 6 ou 12 rondes (résultats de p^6 et p^{12}). Voici les résultats que l'on obtient pour cette simulation en prenant en entrée les valeurs initiales du sujet :

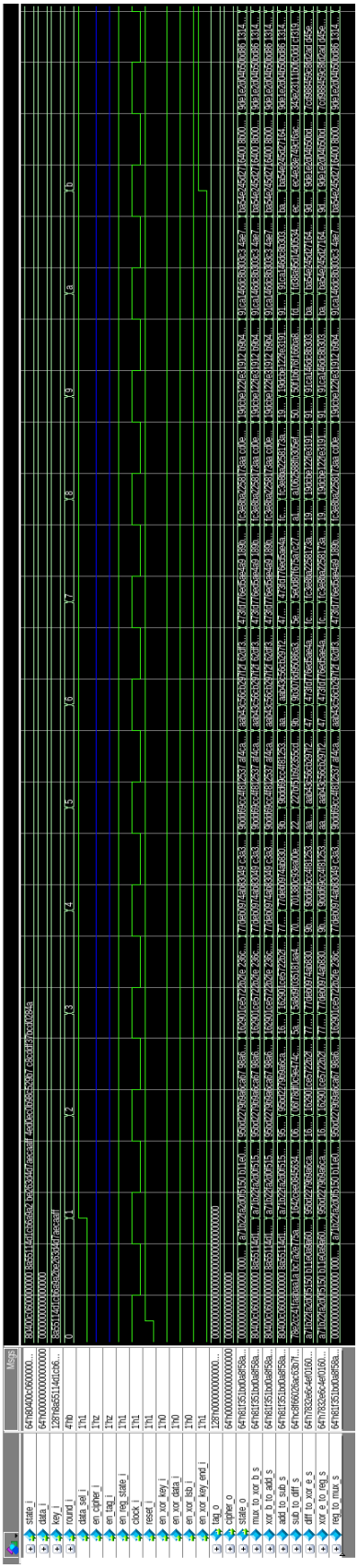


FIGURE 16 – Résultat de la simulation de la permutation finale complète

On obtient bien les résultats souhaités pour nos sorties.

3.2.5 Machine d'états

La machine d'état pilote tout le fonctionnement de l'algorithme de chiffrement en activant ou désactivant l'ensemble des signaux de contrôle, que ce soit pour la permutation finale complète ou bien les compteurs (compteur de ronde et compteur de bloc). On peut bien le percevoir sur le schéma suivant :

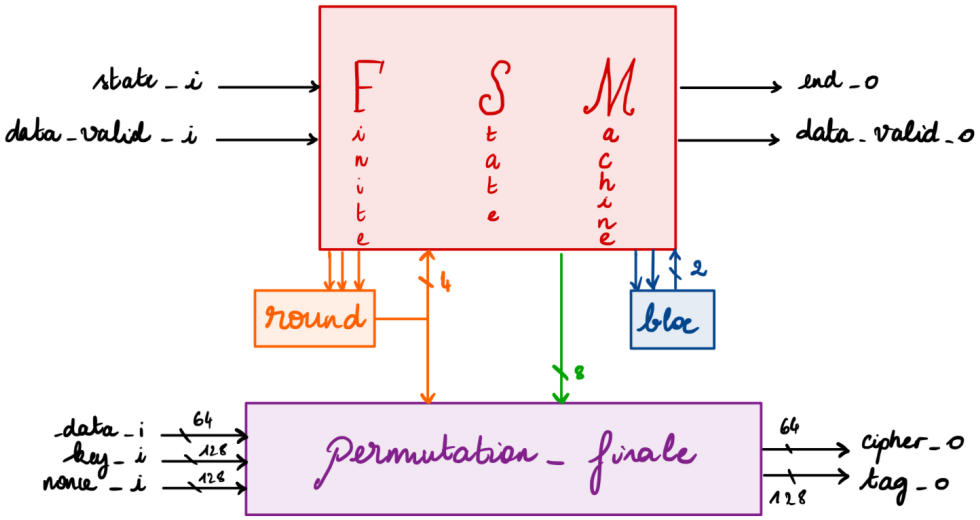


FIGURE 17 – Schéma de l'architecture globale de l'algorithme de chiffrement ASCON-128

La machine d'état base son fonctionnement sur un découpage de l'agorithme de chiffrement en 4 étapes :

1. Initialisation (init)
2. Donnée associée (da)
3. Texte en clair (cipher)
4. Finalisation (final)

On peut retrouver la découpe de ces états sur le schéma complet du chiffrement de l'algorithme ASCON-128 :

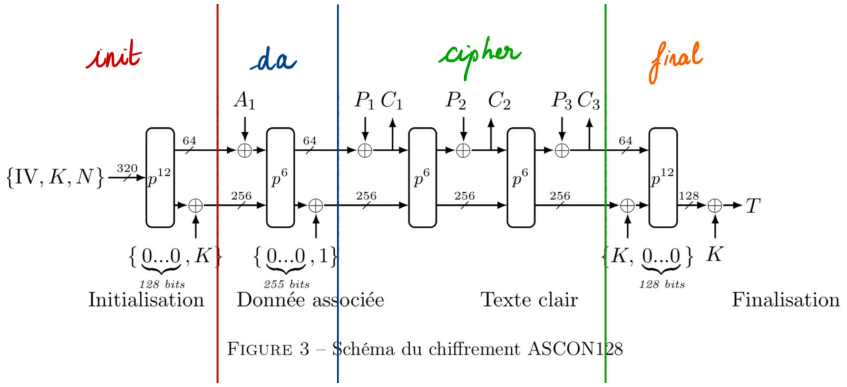


FIGURE 3 – Schéma du chiffrement ASCON128

FIGURE 18 – Étapes de chiffrement

On peut également réaliser un graphe d'états de la machine d'états (qui est au passage une machine de Moore) associé à ces étapes. Par soucis de méthode, je n'ai pu réaliser que l'initialisation :

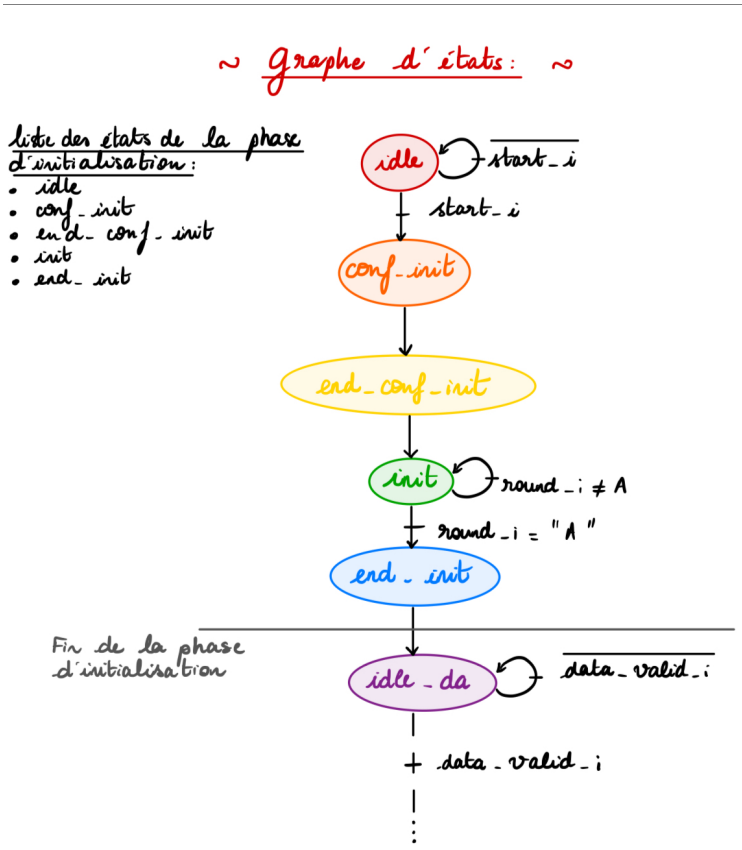


FIGURE 19 – Graphe d'état de la machine de Moore

Ceci étant fait, on peut implémenter cette machine d'états de Moore dans un fichier SystemVerilog. On y déclare les entrées et sorties puis on définit la valeur des sorties pour chaque état de l'initialisation (idle, conf_init, end_conf_init, end_init). J'ai décidé de réaliser un testbench pour cette initialisation de la machine d'état afin d'essayer de visualiser les sorties pour chaque état. Voici les résultats obtenus pour la simulation :

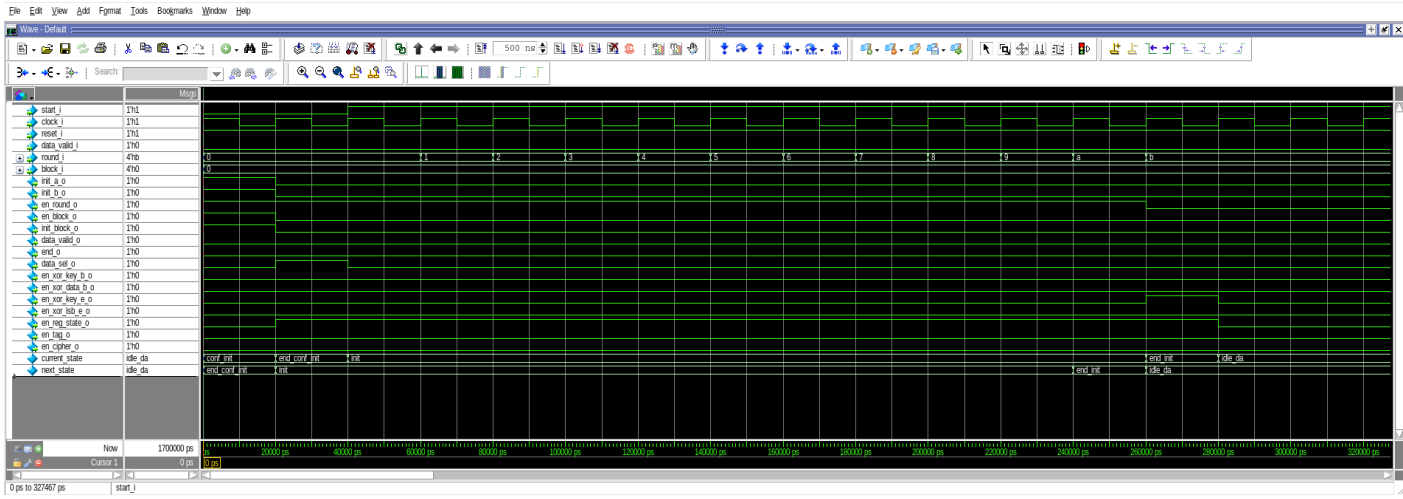


FIGURE 20 – Résultat de la simulation de la machine de Moore

3.2.6 Implémentation de l'architecture complète

Tous les blocs de l'architecture complète de l'algorithme de chiffrement ASCON-128 ont été définis et implémentés en langage SystemVerilog. Il ne reste plus qu'à les relier entre eux comme cela a été défini sur le schéma de l'architecture globale que l'on a vu précédemment :

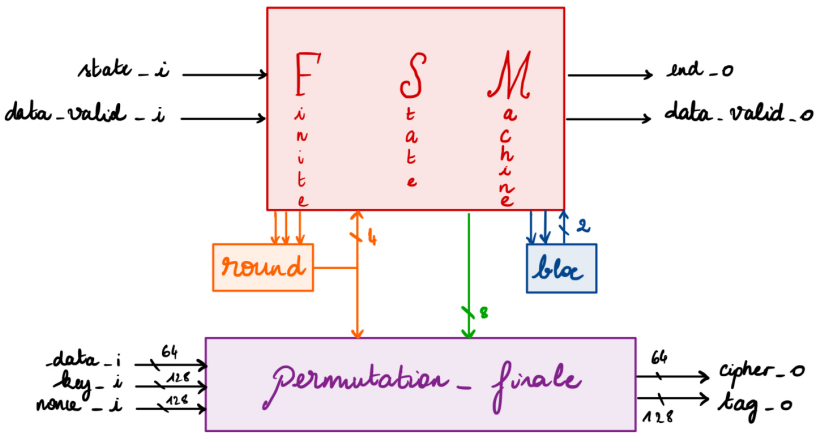


FIGURE 21 – Schéma de l'architecture globale de l'algorithme de chiffrement ASCON-128

Le fichier SystemVerilog "ascon_top.sv" s'occupe de réaliser l'instanciation et la liaison de chacun des blocs. Comme on le voit sur le schéma, il s'agit d'établir les liens entre les modules de la machine d'état, du compteur de rondes(roud), du compteur de blocs(bloc) et de la permutation finale.
On crée par la suite un testbench pour vérifier la validité de cette implémentation complète de l'algorithme de chiffrement ASCON-128 en prenant en entrée les valeurs données par le sujet. Pour observer la valeur de la donnée issue du chiffrement, il faut lire la valeur de la sortie cipher_o. La sortie end_o indique la fin du chiffrement et la sortie tag_o nous indique l'authenticité des données chiffrées.
Étant donné que je n'ai pas implémenté la totalité de la machine d'état, mon algorithme de chiffrement n'est pas complètement abouti et il m'est impossible de vérifier la totalité de mes résultats avec les valeurs fournies par le sujet. J'ai tout de même réalisé la simulation sur ModelSim et voici les résultats que j'ai obtenus :

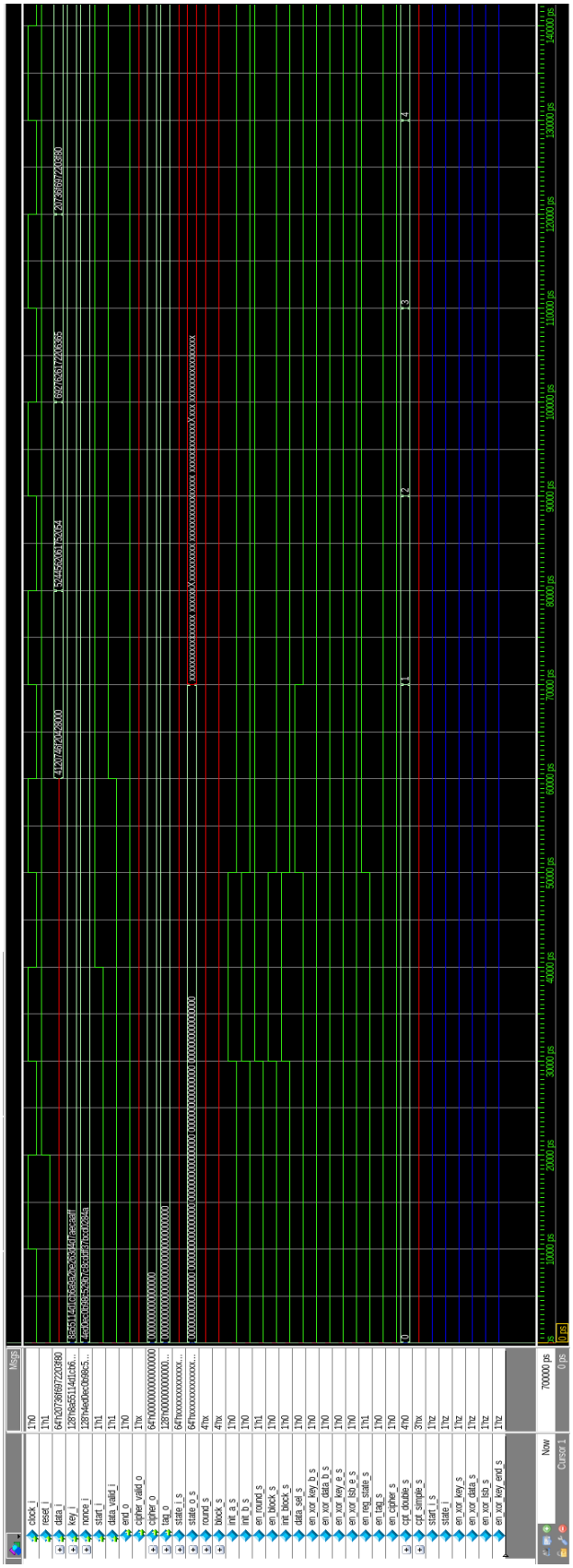


FIGURE 22 – Résultat de la simulation de l'algorithme complet ASCON-128

4 Difficultés liées au projet

Ce projet de conception de l'algorithme de chiffrement ASCON-128 m'a permis d'implémenter un grand nombre de modules dans un langage de description matérielle qui est tout nouveau pour moi. Néanmoins, cette tâche a été loin d'être facile ; il n'a pas toujours été évident pour moi d'appliquer en langage SystemVerilog les principes et idées que j'avais compris de l'algorithme de chiffrement ASCON-128. En effet, c'est un langage tout nouveau pour moi dont le fonctionnement ne me paraissait pas très intuitif au début, d'autant plus que je n'avais jamais utilisé de langage de description auparavant.
J'ai également eu des difficultés à comprendre le fonctionnement de la machine d'état, même en me limitant à l'initialisation ; j'ai eu du mal à définir correctement les différents paramètres associés à chaque état de la phase d'initialisation et j'ai du réaliser un grand nombre de schémas pour clarifier les points me paraissant flous.
Une autre difficulté que j'aimerais mettre en avant est la précision et la minutie dont il faut faire preuve lors de l'instanciation de certains modules et de la liaison entre eux. Par exemple, j'ai du me reprendre à plusieurs fois lors de l'implémentation des fichiers "permutation_finale.sv" et "ascon_top.sv", notamment pour bien associer à chaque module les bonnes entrées et les bonnes sorties, sans les confondre avec d'autres signaux aux appellations similaires. C'est dans ce contexte que l'organisation des codes et l'utilisation d'une bonne syntaxe pour les différents signaux prend toute son importance.

5 Conclusion

Grâce au projet ASCON-128, j'ai eu l'opportunité de m'initier au domaine de la protection et du chiffrement des données. J'ai trouvé intéressant de concevoir un système numérique tout en découvrant un nouveau langage de description, le langage SystemVerilog. Cette expérience m'a permis de découvrir de nouvelles façons de coder et d'avoir une autre vision de la conception numérique.
Ce projet m'a également permis de mettre en œuvre les compétences que j'ai acquises lors du semestre précédent en électronique numérique, pour réaliser un projet original et qui a une application directe dans le domaine de la sécurité et de la protection de données. J'ai pu découvrir de nouveaux enjeux ; il m'a paru intéressant de me surpasser et de me donner de la peine à comprendre un nouveau langage et à le maîtriser même si cela n'a pas toujours été facile. J'ai également pu comprendre des notions algorithmiques complexes, ce qui est très valorisant pour moi.
Cependant, je suis un peu déçu de ne pas avoir pu coder toute la machine d'état, ce qui fait que mon projet n'est pas parfaitement abouti et que je n'ai pas pu obtenir un chiffrement complet et fonctionnel. Cela est un peu frustrant car je n'ai pas pu vérifier les résultats attendus dans le sujet.
Néanmoins, j'ai trouvé cela très plaisant de coder dans un contexte plus réel et concret que celui des algorithmes classiques étudiés en cours d'algorithmique et programmation.
Je remercie mon professeur Jean-Baptiste Rigaud pour la qualité des enseignements qu'il nous a offerts, ainsi que pour l'aide et les conseils qu'il nous a apportés en fournissant des codes très utiles et des schémas détaillés. Sans ces derniers, il m'aurait été impossible de comprendre certains concepts clés du projet.